

BRAC'T's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 3

Roll no: 28 PRN No: 12411933 Name of Student: **Pragati S. Rakhunde**

Subject Name : Deep Learning

Assignment 3:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

- Import the required Python libraries such as TensorFlow, NumPy, Scikit-learn, and Matplotlib.
- Load the Breast Cancer dataset using `load_breast_cancer()` from Scikit-learn.
- Separate the dataset into input features (x) and target labels (y).
- Split the dataset into training and testing sets using `train_test_split()`.
- Standardize the input features using `StandardScaler` to improve training performance.
- Convert the training and testing data into TensorFlow tensors.
- Define different learning rates (e.g., 0.1, 0.01, 0.001, and 0.0001) and set the number of training epochs.
- For each learning rate:
 - Create a new Sequential Neural Network model.
 - Add one hidden layer with ReLU activation.
 - Add one output neuron with Sigmoid activation for binary classification.
 - Initialize the SGD optimizer with the current learning rate.
 - Use Binary Cross-Entropy as the loss function.

- Train the model using TensorFlow's `GradientTape`:
 - Perform a forward pass to compute predictions.
 - Calculate the Binary Cross-Entropy loss.
 - Compute gradients of the loss with respect to trainable parameters.
 - Update the model weights using the SGD optimizer.
 - Repeat the process for the specified number of epochs.
- After training, predict the labels for the test dataset.
- Convert predicted probabilities into binary class labels using a threshold of 0.5.
- Calculate the classification accuracy on the test dataset.
- Record the final loss and accuracy for each learning rate.

Code :

```
import tensorflow as tf
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
# Load Breast Cancer Dataset
```

```
breast = load_breast_cancer()
X = breast.data
y = breast.target
print("Dataset Shape :", X.shape)
print("Target Classes :", np.unique(y))
```

```
# Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split( X,
y,test_size=0.2,random_state=42, stratify=y)
```

```
# Feature Scaling
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Convert to Tensor
```

```
X_train = tf.convert_to_tensor(X_train, dtype=tf.float32)
X_test = tf.convert_to_tensor(X_test, dtype=tf.float32)
y_train = tf.convert_to_tensor(y_train.reshape(-1,1), dtype=tf.float32)
y_test = tf.convert_to_tensor(y_test.reshape(-1,1), dtype=tf.float32)
```

```
# Hyperparameters
```

```
learning_rates = [0.1, 0.01, 0.001, 0.0001]
epochs = 1000
```

```
# Train Model for Different Learning Rates
```

```
for lr in learning_rates:
```

```
    print("\n" + "="*60)
    print("Learning Rate :", lr)
    print("="*60)
```

```
    # Create NEW model
```

```
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(32, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])
```

```
    optimizer = tf.keras.optimizers.SGD(learning_rate=lr)
    loss_fn = tf.keras.losses.BinaryCrossentropy()
```

```
    for epoch in range(epochs):
```

```
        with tf.GradientTape() as tape:
            predictions = model(X_train, training=True)
            loss = loss_fn(y_train, predictions)
```

```
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))
```

```
        if (epoch + 1) % 100 == 0:
```

```
            print(f"Epoch {epoch+1}/{epochs}, Loss : {loss.numpy():.5f}")
```

```
# Test Accuracy
```

```
pred = model(X_test, training=False)
pred_class = tf.cast(pred > 0.5, tf.float32)
accuracy = tf.reduce_mean(tf.cast(tf.equal(pred_class, y_test), tf.float32))
print("Test Accuracy :", round(float(accuracy) * 100, 2), "%")
```

Output :

```
=====
Learning Rate : 0.1
=====
Epoch 100/1000, Loss : 0.08767
Epoch 200/1000, Loss : 0.06214
Epoch 300/1000, Loss : 0.05124
Epoch 400/1000, Loss : 0.04420
Epoch 500/1000, Loss : 0.03905
Epoch 600/1000, Loss : 0.03514
Epoch 700/1000, Loss : 0.03183
Epoch 800/1000, Loss : 0.02893
Epoch 900/1000, Loss : 0.02647
Epoch 1000/1000, Loss : 0.02434
```

```
=====
Learning Rate : 0.01
=====
Epoch 100/1000, Loss : 0.28587
Epoch 200/1000, Loss : 0.18774
Epoch 300/1000, Loss : 0.14773
Epoch 400/1000, Loss : 0.12619
Epoch 500/1000, Loss : 0.11231
Epoch 600/1000, Loss : 0.10247
Epoch 700/1000, Loss : 0.09498
Epoch 800/1000, Loss : 0.08905
Epoch 900/1000, Loss : 0.08421
Epoch 1000/1000, Loss : 0.08018
```

```
=====
Learning Rate : 0.001
=====
Epoch 100/1000, Loss : 0.78842
Epoch 200/1000, Loss : 0.65793
Epoch 300/1000, Loss : 0.56625
Epoch 400/1000, Loss : 0.49992
Epoch 500/1000, Loss : 0.45057
Epoch 600/1000, Loss : 0.41243
Epoch 700/1000, Loss : 0.38187
Epoch 800/1000, Loss : 0.35682
Epoch 900/1000, Loss : 0.33581
Epoch 1000/1000, Loss : 0.31786
```

```
=====
Learning Rate : 0.0001
=====
Epoch 100/1000, Loss : 0.62919
Epoch 200/1000, Loss : 0.61737
Epoch 300/1000, Loss : 0.60604
Epoch 400/1000, Loss : 0.59515
Epoch 500/1000, Loss : 0.58471
Epoch 600/1000, Loss : 0.57466
Epoch 700/1000, Loss : 0.56502
Epoch 800/1000, Loss : 0.55574
Epoch 900/1000, Loss : 0.54678
Epoch 1000/1000, Loss : 0.53816
```

Final Result :

```
Test Accuracy : 77.19 %
```